DRFusionTrack: Underwater Dead Reckoning and Sensor Fusion for Garmin Wearables

MSc Panagiotis Nikolaou

***Abstract*—DRFusionTrack is a Garmin Connect IQ watch app for underwater navigation, motion tracking, and FIT logging. It combines pressure-derived depth, motion estimation, GPS quality gating, and local dead reckoning to build a stable underwater track when GPS is weak, noisy, or unavailable. The implementation is tuned for the realities of wrist-mounted marine sensing: shallow transitions, intermittent fixes, and the need to remain stable while switching between underwater and surface states. The current project state reflects the final implementation: pressure is the primary depth source, the first valid reading becomes the active surface baseline, and GPS corrections are applied only when they are fresh and reliable enough to improve the local estimate.**



Fig. 1: DRFusionTrack running in simulation on the smartwatch UI.

***Index Terms*—Garmin Connect IQ, underwater navigation, dead reckoning, pressure-based depth, GPS fusion, FIT logging, wearable sensor fusion**

I. Introduction

Garmin devices expose accelerometer, gyroscope, magnetometer, barometer, and GPS data through Connect IQ, but they do not provide a turnkey underwater navigation engine for dive logging or local route reconstruction. DRFusionTrack addresses this gap with a custom watch app that estimates motion, heading, depth, and local East/North position while remaining robust when GPS quality degrades or disappears.

The central design choice is a conservative state estimator: it propagates the motion model from IMU data and applies GPS corrections sparsely when the fix is good enough to improve the estimate. This is essential because GPS over water is often noisy and multipath-prone, and the wrist form factor amplifies motion jitter. The result is a practical marine tracking system that remains stable during shallow dives, weak fixes, and short sensor dropouts.

II. Project Goals

The project aims to provide a watch-app solution that can:

- estimate current depth from barometric pressure,
- classify underwater and surface states without unstable toggling,
- estimate motion and heading from the best available sensor source,
- continue local motion tracking even when GPS is weak or absent,
- export a rich FIT stream for later analysis and route reconstruction.

These goals are consistent with the final implementation in the app source, where the main runtime logic remains conservative and sensor-driven rather than overly aggressive.

III. System Architecture

The runtime is organized into a few major components:

- `gsensrosView.mc`: main runtime controller, sensor polling, dive-state logic, UI, and FIT write path.
- `DeadReckoning.mc`: fallback DR model for local track propagation when the filtered estimate is not available.
- `KalmanDR.mc`: local East/North estimator that advances the motion state and applies GPS corrections.
- `NitrogenModel.mc`: a simple exposure-risk model for session logging and UI output.

The app follows a timer-driven loop: read sensor and GPS inputs, update depth and motion state, update the local DR estimate, and write FIT values into the active session.

IV. Depth and Dive-State Detection

The key improvement in the final implementation is pressure-first depth estimation. The app uses ambient pressure as the primary depth source and initializes the live surface reference from the first valid pressure sample. This

avoids the earlier failure mode in which realistic shallow values were discarded as startup noise.

The model is:

$$\text{depth}_m \approx \frac{P_{ambient} - P_{surface}}{PA_PER_METER_SEAWATER} \quad (1)$$

with the seawater coefficient:

$$PA_PER_METER_SEAWATER \approx 10050\ Pa/m. \quad (2)$$

Altitude is treated as a secondary validation signal rather than the primary depth source. This means that shallow real values are preserved and the app remains stable when the watch is near the surface or when the altitude path is noisy.

The state machine also uses hysteresis and a minimum-duration gate so that small pressure spikes do not trigger false transitions between underwater and surface states. The preferred policy is to maintain a shallow threshold near real recorded values and require a sustained return before the app exits underwater mode.

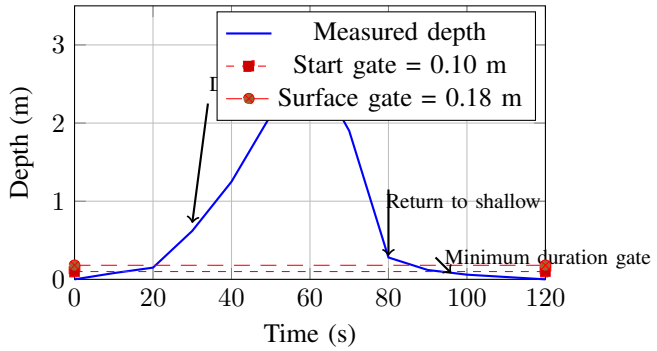


Fig. 2: Representative shallow-water dive transition logic with a start gate and surface-return hold.

V. Motion, Heading, and Sensor Fusion

The app uses accelerometer, gyroscope, and magnetometer inputs to estimate motion and heading. It maintains a motion-confidence signal and an adaptive baseline from recent sensor history, then applies hysteresis before declaring the user moving or stationary.

Heading is selected using the strongest valid source in order of reliability:

1) true heading if available,
2) magnetometer heading after calibration and bias correction,
3) gyro-integrated heading when motion remains consistent,
4) previous valid heading as a fallback.

Speed is estimated from GPS when it is fresh and usable; otherwise the app falls back to motion-confidence logic and underwater/surface thresholds. This allows the DR model to continue updating even when the fix is weak or intermittent.

VI. Dead Reckoning and GPS Correction

The local navigation model maintains a 4-state East/North estimate:

$$\mathbf{x} = \begin{bmatrix} E \\ N \\ v_E \\ v_N \end{bmatrix} \quad (3)$$

where E and N are local displacements and v_E, v_N are their velocities. The estimator propagates the state with the best available speed and heading, then applies a GPS correction only when the fix is fresh and good enough to improve the estimate.

This is essential in underwater use because GPS over water is noisy and multipath-prone. The app therefore uses conservative covariance values and avoids aggressive updates. The local track remains continuous even when the external fix is sparse, delayed, or temporarily unavailable.

The coordinate system is local to the session reference, which allows the app to build a continuous East/North track and then reconstruct synthetic latitude and longitude values for FIT logging. This gives a useful post-session path even when only intermittent GPS observations are available.

VII. FIT Logging and Data Export

DRFusionTrack writes custom FIT developer fields at a regular interval during recording. These fields include:

- DR East and North displacement,
- depth,
- speed estimate,
- heading,
- motion and moving state,
- GPS fix state,
- underwater boolean,
- synthetic DR latitude and longitude values.

The custom records are designed for later analysis, review, and comparison against external dive or GPS logs. The export is intentionally compact but rich enough to reconstruct the session behavior in software after the watch session ends.

VIII. Validation and Build Status

The project was validated through the local Garmin Connect IQ build flow. The current app compiles successfully against the installed SDK and developer key and remains consistent with the final source implementation.

The active project build check is:

```
cd /home/panagiotis/Documents/gsensors/gsensros
./run_simulation.sh --build-only
```

This confirms the current state of the app compiles successfully with the available toolchain and that the active app name and references remain aligned with the DRFusionTrack branding.

IX. Limitations and Practical Use

This app is intended as a custom underwater tracker and analysis tool rather than a certified dive computer. Its limitations are important:

- the local DR estimate is 2-D; vertical motion is handled separately by depth,

- underwater GPS is noisy and multipath-prone,
- motion heuristics remain approximate when the fix is poor,
- synthetic track values are reconstructed from local coordinates and are best used for post-session review.

Despite these limitations, the app is well suited to underwater session logging, motion analysis, and real-world evaluation of pressure and GPS-aided navigation on Garmin hardware.

X. Conclusion

DRFusionTrack represents the verified state of the project: pressure-first depth estimation, conservative motion gating, local dead reckoning, GPS-assisted correction, and practical FIT logging for underwater sessions. The design is intentionally cautious and robust under noisy sensor conditions, making it a useful foundation for ongoing development, tuning, and on-device validation.